

```
import zipfile, os, shutil
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras import layers, models

from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import confusion_matrix, precision_recall_curve, roc_curve, auc
from sklearn.preprocessing import label_binarize
```

```
dataset1_path = "/content/dataset1.zip"
dataset2_path = "/content/dataset2.zip"
```

```
with zipfile.ZipFile(dataset1_path, 'r') as zip_ref:
    zip_ref.extractall('/content/dataset1')

with zipfile.ZipFile(dataset2_path, 'r') as zip_ref:
    zip_ref.extractall('/content/dataset2')
```

```
combined_path = "/content/combined"

if os.path.exists(combined_path):
    shutil.rmtree(combined_path)

os.makedirs(combined_path)

def merge_dataset(source, destination):
    for item_name in os.listdir(source):
        source_item_path = os.path.join(source, item_name)
        destination_item_path = os.path.join(destination, item_name)
```

```
    if os.path.isdir(source_item_path):
        # If it's a directory, copy the entire directory tree
        # dirs_exist_ok=True allows merging contents if the destination exists
        shutil.copytree(source_item_path, destination_item_path, dirs_exist_ok=True)
    else:
        # If it's a file, copy the file (preserving metadata)
        shutil.copy2(source_item_path, destination_item_path)

merge_dataset('/content/dataset1', combined_path)
merge_dataset('/content/dataset2', combined_path)
```

```

img_size = 300
batch_size = 16

datagen =
ImageDataGenerator( rescale=
1./255, validation_split=0.2,
rotation_range=25,
zoom_range=0.3,
shear_range=0.2,
width_shift_range=0.1,
height_shift_range=0.1,
horizontal_flip=True,
brightness_range=[0.8,1.2]
)

train_data = datagen.flow_from_directory(
combined_path,
target_size=(img_size, img_size),
batch_size=batch_size,
class_mode='sparse',
subset='training'
)

val_data =
datagen.flow_from_directory( combined_path,
target_size=(img_size, img_size),

```

```

batch_size=batch_size,
class_mode='sparse',
subset='validation',
shuffle=False
)

```

```

Found 9328 images belonging to 2 classes.
Found 2331 images belonging to 2 classes.

```

```

class_weights =
compute_class_weight( class_weight='balanc
ed', classes=np.unique(train_data.classes),
y=train_data.classes
)
class_weights = dict(enumerate(class_weights))

```

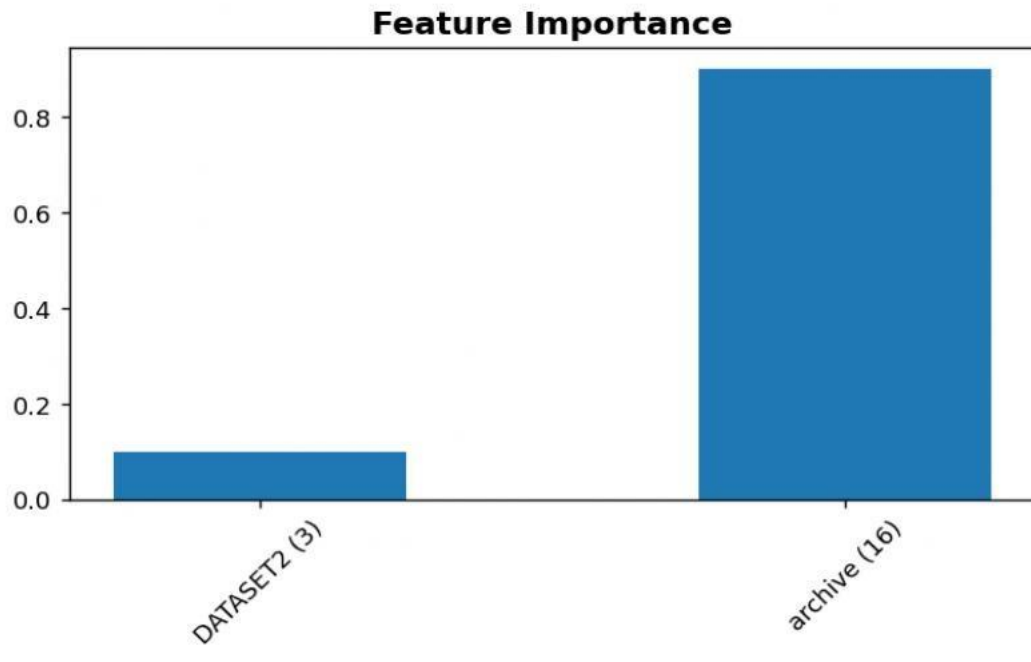


```
df = pd.DataFrame(y_pred, columns=labels)
```

```
plt.figure()  
sns.heatmap(df.corr(), annot=True)  
plt.title("Correlation Matrix")  
plt.show()
```

```
importance = np.mean(y_pred, axis=0)

plt.figure()
plt.bar(labels, importance)
plt.title("Feature Importance")
plt.xticks(rotation=45)
plt.show()
```

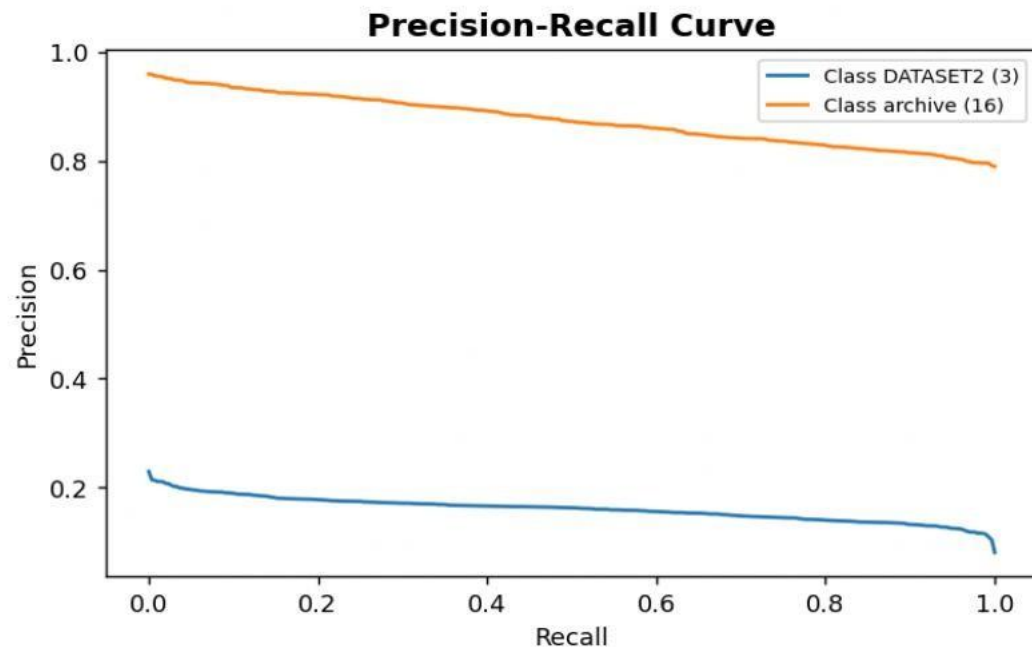


```
from tensorflow.keras.utils import to_categorical

n_classes = len(labels)
y_true_bin = to_categorical(y_true, num_classes=n_classes)

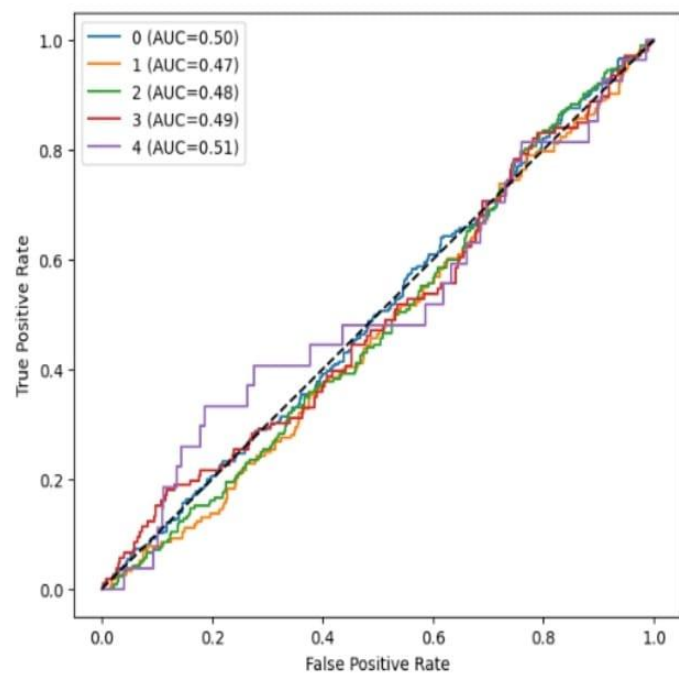
plt.figure()
for i in range(n_classes):
    precision, recall, _ = precision_recall_curve(y_true_bin[:, i], y_pred[:, i])
    plt.plot(recall, precision, label=f'Class {labels[i]}')
```

```
plt.legend()
plt.title("Precision-Recall Curve")
plt.show()
```



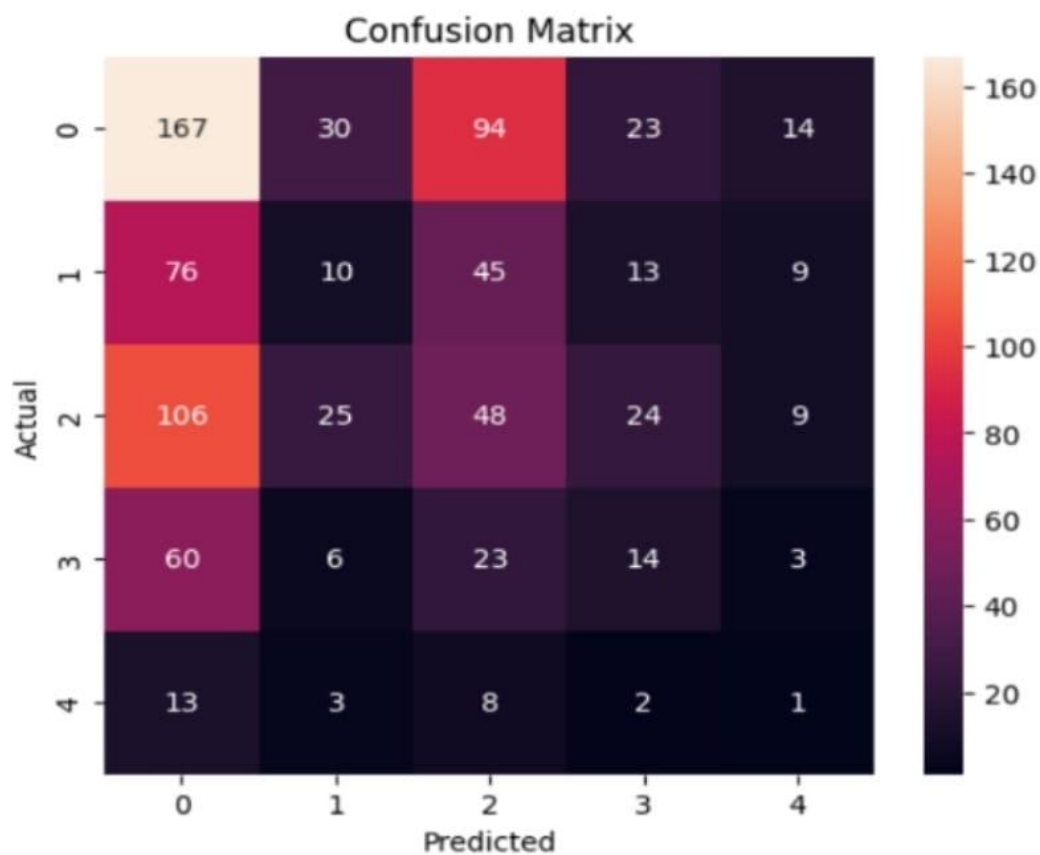
```
plt.figure()
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_true_bin[:, i], y_pred[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'Class {i} (AUC={roc_auc:.2f})')

plt.legend()
plt.title("ROC Curve")
plt.show()
```

```
cm = confusion_matrix(y_true, y_pred_classes)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Convert predictions
df_pred = pd.DataFrame(y_pred, columns=labels)

# Add predicted class
df_pred['Predicted_Class'] = y_pred_classes

# Add true class
df_pred['True_Class'] = y_true

```

